NFP Predictor Pipeline

Institutional-Grade Non-Farm Payrolls Forecasting with Point-in-Time Correctness

Dhruv Kohli

March 16, 2026

Agenda

1. The Problem
2. System Architecture
3. Point-in-Time Correctness
4. Data Sources
5. Feature Selection Engine
6. Training Pipeline
7. Variance Enhancement Stack
8. Metrics & Evaluation
9. Model Variants & Output

What Are Non-Farm Payrolls?

- The **U.S. Non-Farm Payrolls (NFP)** report is the single most-watched economic release globally
- Published by the **Bureau of Labor Statistics (BLS)** on the first Friday of each month
- Measures the **month-over-month change** in total non-farm employment (thousands)
- Two variants:
  - **NSA** (Non-Seasonally Adjusted): Raw count with calendar effects intact
  - **SA** (Seasonally Adjusted): BLS removes recurring seasonal patterns
- Our target: predict the **once-revised MoM change** (from the M+1 FRED snapshot)

Target Definition

$$y_{\text{mom}}^{(t)} = \text{NFP}_t^{\text{revised}} - \text{NFP}_{t-1}^{\text{revised}}$$

where $\text{NFP}_t^{\text{revised}}$ is the once-revised employment level from the $M{+}1$ vintage.

Why This Is Hard: Four Structural Challenges

1. Aggressive Revisions

BLS initial releases are heavily revised in subsequent months. Training on "finalized" data creates **lookahead bias**: assuming access to clean data that didn't exist in real-time.

2. Asynchronous Availability

Indicators arrive at different frequencies (daily, weekly, monthly) with varying lags:

- FRED employment: $\sim$5 days
- ADP: $\sim$2 days before NFP
- NOAA storm data: $\sim$75 days late

3. Regime Non-Stationarity

Economic relationships break during crises. Relationships stable during the Great Moderation (pre-2008) decoupled or **inverted** during GFC and COVID.

4. High-Dimensional Instability

- $p \gg n$ problem: 49,000+ features vs. $\sim$1,000 rows
- Thousands of spurious correlations
- Standard ML will catastrophically overfit

Five-Phase Architecture

Phase 1
Raw Ingestion
6 data sources

FRED, ADP, NOAA, Unifier, Prosper

→

Phase 2
PIT Shielding
Monthly snapshots

release_date < nfp_release_date

→

Phase 3
Dim. Reduction
6-stage selection

15k → ∼80 features

→

Phase 4
Walk-Forward
Expanding window

LightGBM + Optuna

→

Phase 5
Analyst Output
Scorecard + SHAP

CSV, HTML, PNG, SHAP

Data Sources

- 159 FRED employment series
- 10 FRED exogenous indicators
- ADP private payrolls
- NOAA storm damage data
- 11 Unifier/LSEG indicators
- Prosper consumer surveys

Selection Pipeline

- Stage 0: Variance filter
- Stage 1: Dual filter (corr + LightGBM)
- Stage 2: Boruta shadow features
- Stage 3: Vintage stability
- Stage 4: Cluster redundancy
- Stages 5–6: Optional (SFS)

Training

- Expanding window backtest
- Dynamic reselection (every 24mo)
- Optuna composite tuning
- NSA: 5-stage enhancement
- SA: enhancement exempt
- Kalman + AccelOverride fusion

The Fundamental Constraint

Point-in-Time Rule (Applies Everywhere)

$$\forall \text{ feature } X_i \text{ mapped to month } t: \quad \texttt{feature_release_date}(X_i) < \texttt{nfp_release_date}(t)$$

Strict inequality ($<$, not $\leq$): data published on the exact NFP release day is *excluded*.

PIT Enforcement by Source

Source	Mechanism
FRED Empl.	ALFRED vintage `realtime_start`
FRED Exog.	NFP-windowed weekly aggregation
ADP	`release_date < nfp_release_date`
NOAA	month-end + 75 day lag model
Unifier	Median lag repair for missing `first_release_date`
Prosper	`release_date < nfp_release_date`

The Unifier Bug

When `first_release_date` is missing, Unifier fills `last_revision_date` with *today's timestamp*—making 2015 data appear available today.

Fix: Compute per-series `median_lag_days` from valid observations, use that instead. Never trust `last_revision_date` when `first_release_date` is NULL.

PIT: Weekly Data Aggregation

Problem

Weekly claims data (CCNSA, CCSA) cannot be aggregated by calendar month—some weeks span month boundaries, and claims published *after* the NFP release would leak.

Solution: NFP-Windowed Bucketing

```
def aggregate_weekly_to_monthly_nfp_based(weekly_df, nfp_map):
    for each month M:
        nfp_prev = nfp_release_date(M - 1)
        nfp_curr = nfp_release_date(M)
        # Only include data RELEASED between prev and curr NFP
        mask = (weekly_df.release_date >= nfp_prev) &
               (weekly_df.release_date < nfp_curr)
        monthly_agg[M] = weekly_df[mask].mean()
    return monthly_agg
```

A claim reported on January 15th (after the January 10th NFP release) goes into **February's bucket**—matching what an analyst would actually have in hand.

PIT: FRED Release Date Imputation

For historical data where `realtime_start` is unavailable (pre-2009), two strategies:

Simple (First-Release Files)

```
def impute_simple(month):
    # First Friday of the month
    day_1 = date(year, month, 1)
    weekday = day_1.weekday()
    offset = (4 - weekday) % 7 # Friday
    return day_1 + offset
```

Complex (Last-Release Files)

3-tier gate system:

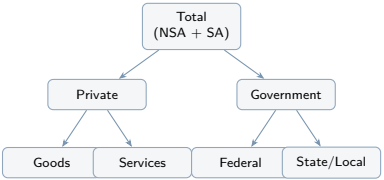
- **Option 1:** Extended backfill for very old data (pre-2009, no metadata)
- **Option 2:** Intermediate backfill for partial metadata
- **Option 3:** Closest candidate respecting snapshot timing

Data Source Overview

Source	API / Origin	Frequency	Features	History
FRED Employment	FRED / ALFRED	Monthly	∼11,600	∼1948
FRED Exogenous	FRED + Yahoo Finance	Daily/Weekly	∼50	∼1990
ADP Employment	investing.com	Monthly	∼15	∼2001
NOAA Storm Events	NCEI (gzip CSV)	Event-level	∼30	∼1996
Unifier / LSEG	Unifier API	Monthly	∼100	varies
Prosper Surveys	Prosper API	Monthly	∼200	∼2009
Total			**∼15,500**	

- All sources produce monthly **PIT-safe parquet snapshots** stored in decade-based directory structure
- Path convention: `base_dir/decades/{decade}s/{year}/YYYY-MM.parquet`
- Feature count after ETL-time feature selection: ∼15,000 (all-features mode; selection deferred to training)

FRED Employment: 159-Series Hierarchy



Full hierarchy extends 7 levels deep into sub-industries (Manufacturing, Construction, Mining, Health Care, Leisure, etc.)

9 Features Per Series ($\times$ 159 Series = 1,431)

Feature	Formula
`_latest`	Raw level L_t
`_MoM`	$L_t - L_{t-1}$
`_MoM_pct`	$(L_t - L_{t-1})/L_{t-1}$
`_3m`	$L_t - L_{t-3}$
`_6m`	$L_t - L_{t-6}$
`_YoY`	$L_t - L_{t-12}$
`_12m_pct`	$(L_t - L_{t-12})/L_{t-12}$
`_rolling_3m`	$\frac{1}{3}\sum_{k=0}^{2} \Delta L_{t-k}$
`_volatility`	$\text{std}(\Delta L_{t:t-2})$

Data sourced via **ALFRED vintage API** to preserve `realtime_start` metadata for PIT correctness.

FRED Exogenous: Financial & Macro Indicators

10 Indicator Series

Series	FRED ID	Signal
Credit Spreads	BAMLH0A0HYM2	Risk appetite
Yield Curve	T10Y2Y	Recession proxy
Oil Prices	DCOILWTICO	Energy sector
VIX	VIXCLS	Market fear
S&P 500	Yahoo Finance	Equity market
Fin. Stress	STLFSI4	Systemic risk
Econ. Index	WEI	Weekly GDP proxy
Claims (NSA)	CCNSA	Labor slack
Claims (SA)	CCSA	Labor slack
Ind. Prod	—	Output signal

Binary Regime Indicators

NOT differenced—binary nature makes transforms meaningless:

- `VIX_panic_regime`: VIX > 40
- `VIX_high_regime`: VIX > 25
- `SP500_bear_market`: drawdown $> 20\%$
- `SP500_crash_month`: return $< -10\%$
- `SP500_circuit_breaker`: drop $> 7\%$

API Resilience

Exponential backoff: 2s, 4s, 8s. Thread-safe rate limit: 0.8s/request.

ADP, NOAA, Unifier, Prosper

ADP Employment

- Source: investing.com REST API
- Private-sector payroll change (thousands)
- Independent from BLS establishment survey
- PIT: `release_date < nfp_release_date`
- Features: `pct_change` + lean `compute_all_features` (no symlog)

Unifier / LSEG (11 Indicators)

- ISM Mfg & Non-Mfg, Consumer Confidence
- Housing Starts, Retail Sales, Industrial Production
- Avg Weekly Hours, Avg Hourly Earnings
- Empire State Mfg, UMich Expectations
- **NFP Consensus Poll** (economists' mean forecast)
- Zero-centered handling for Empire State

NOAA Storm Events

- CPI-adjusted property & crop damage
- State→national: **employment-weighted** aggregation
- 75-day lag after month-end (NCEI processing)
- Hurricane in Texas (large employment) > same storm in Wyoming

Prosper Consumer Surveys

- Consumer Mood, Spending Forecast
- Employment expectations by demographic
- **Historical break fix:** Pre-9/2009 "employed" question split into FT+PT post-9/2009; merged into continuous series

Feature Transforms: The SymLog & Derived Feature Chain

Transform Chain (Applied Per Source)

1. `add_symlog_copies()`: $\text{symlog}(x) = \text{sign}(x) \cdot \ln(1 + |x|)$
2. `add_pct_change_copies()`: $\Delta\% = \frac{x_t - x_{t-1}}{x_{t-1}} \times 100$
3. `compute_all_features()`: Lags, rolling stats, momentum

Derived Features Per Series (Full Mode)

Feature	Formula
`level`	x_t
`diff`	$x_t - x_{t-1}$
`diff_zscore_3m`	$(x_t - \bar{x}_{3m})/\sigma_{3m}$
`diff_zscore_12m`	$(x_t - \bar{x}_{12m})/\sigma_{12m}$
`zscore_3m/12m`	Rolling z-score of level
`rolling_mean_3m`	$\frac{1}{3}\sum_{k=0}^{2} x_{t-k}$
`rolling_std_6m`	$\sigma(x_{t:t-5})$
`chg_3m/6m/12m`	$x_t - x_{t-k}$
`lag_1/3/6/12m`	x_{t-k}

Why SymLog?

- Standard log undefined for negatives
- SymLog handles *both* signs
- Preserves zero values: $\text{symlog}(0) = 0$
- Compresses extreme kurtosis
- On NFP MoM data: skewness $-6.5 \to -1.1$, kurtosis $81 \to -0.7$

Lean Mode (ADP, etc.)

Skip symlog (tree-invariant for monotonic transforms). Keep only: level, pct_chg, diff, diff_zscore_3m, rolling stats, chg_3,6,12m.

Revision Features: Capturing Data Revision Dynamics

Motivation

When BLS revises prior months' data, the *pattern* of revisions contains signal—systematic upward revisions suggest strengthening, while large revision volatility suggests measurement uncertainty.

6 Aggregate Revision Features

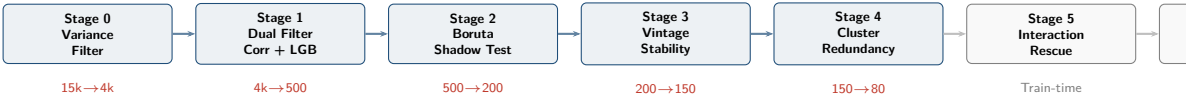
Feature	Formula
`rev_mean`	$\bar{r} = \text{mean}(S_t^{(M)} - S_t^{(M-1)})$
`rev_abs_mean`	$\text{mean}(\lvert r_i \rvert)$
`rev_positive_ratio`	$\text{mean}(r_i > 0)$
`rev_count`	$\#\{r_i \neq 0\}$
`rev_max_abs`	$\max(\lvert r_i \rvert)$
`rev_std`	$\text{std}(r_i)$

Construction

For target month M:

1. Load snapshot at M and snapshot at $M-1$
2. Restrict both to observations **before** $M-1$ cutoff (no lookahead)
3. Compute: $r_i = \text{snap}_M[i] - \text{snap}_{M-1}[i]$
4. Aggregate into 6 statistics

Feature Selection: The 7-Stage Pipeline



Default Pipeline: Stages (0, 1, 2, 3, 4)

- Runs **independently per source** and **per regime**
- ∼5 min per source
- Results cached with 30-day TTL
- Stages 5–6: used in dynamic reselection at train time

Stage Presets

Preset	Stages
Default	(0,1,2,3,4)
Fast	(0,1,4)
Fast+Vintage	(0,1,3,4)
Fast+Boruta	(0,1,2,4)
Full	(0,1,2,3,4,5,6)

Stage 0: Variance Filter + Spearman Pre-Screen

Variance Filter

Remove features where $\geq 97\%$ of non-NaN values are identical.

Two-tier approach (for efficiency on 17k+ features):

1. **Tier 1 (fast):** `nunique()` classification
   - nunique ≤ 1: constant $\rightarrow$ **drop**
   - nunique > 5: variable $\rightarrow$ **keep**
   - nunique $\in [2, 5]$: ambiguous
2. **Tier 2 (exact):** `np.unique()` per ambiguous column; compute mode frequency

Minimum 30 non-NaN values required per column.

Spearman Pre-Screen ($p > 5{,}000$)

When source has $> 5{,}000$ features (FRED Employment at $\sim$17k):

1. Compute vectorized Spearman ρ with target
2. Apply Benjamini–Hochberg FDR correction at $\alpha = 0.30$
3. Drop features with $p_{\text{adj}} > 0.30$

Effect: Reduces Stage 1 input by $\sim$78%, cutting runtime from $\sim$10 min to $\sim$2 min with zero empirical signal loss.

Stage 1: Dual Filter

A) Purged Expanding Correlation

Expanding windows with **3-month purge gap**:

$$\bar{\rho} = \frac{\sum_k \sqrt{n_k} \cdot |\rho_k|}{\sum_k \sqrt{n_k}}$$

where ρ_k = Spearman correlation in window k, n_k = window size.

Purge: 3-month gap between training and evaluation prevents information leakage.

Deduplication via hierarchical clustering:

- Spearman threshold: 0.95
- Agglomerative with average linkage
- Chunked for $> 5{,}000$ features (2-phase: local→merge with shuffling)

B) Random Subspace LightGBM

Captures **non-linear importance** that correlation misses:

```
for each random subspace:
    X_sub = random_sample(features, k)
    model = LightGBM(X_sub, y)
    importance[X_sub] += model.gain
# Rank by cumulative importance
```

LightGBM parameters:

- `objective`: regression (L2)
- `n_estimators`: 100
- `num_leaves`: 31
- `n_jobs`: 1 (macOS OOM safety)

Stages 2–4: Boruta, Vintage Stability, Cluster Redundancy

Stage 2: Boruta

Shadow-feature permutation test (100 runs):

1. Create "shadow" features by shuffling each real feature
2. Train LightGBM on real + shadow
3. Record max shadow importance
4. Feature "confirmed" if it beats shadow max in statistically significant number of runs (binomial test)

Guard: Max 500 features input.

Stage 3: Vintage Stability

Reject features with unstable predictive power across regimes.

6 Hard-coded regimes:

1. Pre-GFC (1998)
2. GFC Shock (2008)
3. Late-Cycle (2015)
4. COVID (2020-03)
5. Inflation (2022-03)
6. AI/Trump (2025-02)

Exponential recency weighting. Default lookback: 3 months (NOAA: 6).

Stage 4: Cluster Redundancy

Collapse collinear survivors:

1. **NaN-aware** Spearman correlation (pairwise non-NaN overlap)
2. Hierarchical clustering
3. Keep one representative per cluster (highest target correlation)

Key: NaN-awareness handles different start dates (FRED ∼1948, ADP ∼2001, Prosper ∼2009).

Master Snapshot Aggregation

7 Source Directories → 1 Wide Parquet Per Month

Source Key	Exec Order
FRED_Employment_NSA	1st (longest)
FRED_Employment_SA	2nd
FRED_Exogenous	3rd
Unifier	4th
Prosper	5th
NOAA	6th
ADP	7th (fastest)

All-Features Mode (Current)

Master snapshots store **all** lean features (∼15,500 per month). No ETL-time selection.

Feature selection is entirely deferred to **train-time dynamic reselection** (2-pass Boruta + cluster redundancy), which selects ∼80 features per backtest window.

Advantage: Every feature is re-evaluated with current data—no permanent information loss from stale ETL decisions.

Target Combos

Target Type	Source	FS Target Mode
NSA	revised	`mom`
SA	revised	`model_aligned`

Caching Architecture

- **Source cache:** per source, 30-day TTL
- **Regime cache:** per cutoff month, 30-day TTL
- **Branch cache:** per target config, 30-day TTL
- Version: `2026-02-24-regime-cache-v1`

Expanding Window Backtest: The Core Loop

```
for target_month in [oldest_backtest_month, ..., latest]:
    # 1. Expanding window: data strictly before target
    X_train, y_train = load_data(before=target_month)

    # 2. Feature engineering (parallel via joblib)
    X_train += calendar_features(target_month)
    X_train += lagged_target_features(y_train)
    X_train += revision_features(target_month)

    # 3. Dynamic reselection (every 24 months, 2-pass)
    if months_since_reselection >= 24:
        features = dynamic_reselect(X_train, y_train) # max 80

    # 4. Optuna tuning (every 12 months, composite objective)
    if months_since_tune >= 12:
        params = optuna_tune(X_train, y_train, target_month)

    # 5. Train LightGBM (85/15 chronological split)
    model = train_lgbm(X_train[features], y_train, params)

    # 6. Variance enhancement (NSA: full stack; SA: exempt)
    prediction = enhance(model, X_test, enhancements)

    # 7. Record result
    results.append(prediction, actual, intervals, baselines)
```

Feature Engineering: Calendar & Survey Week Features

Cyclical Encoding

Preserves December → January proximity (superior to one-hot for trees):

$$\texttt{month_sin} = \sin\left(\frac{2\pi \cdot m}{12}\right)$$

$$\texttt{month_cos} = \cos\left(\frac{2\pi \cdot m}{12}\right)$$

$$\texttt{quarter_sin} = \sin\left(\frac{2\pi \cdot q}{4}\right)$$

$$\texttt{quarter_cos} = \cos\left(\frac{2\pi \cdot q}{4}\right)$$

SA Calendar Filtering

SA series have seasonality stripped by BLS. Only keep: `weeks_since_last_survey`, `is_5_week_month`, `year`.

BLS Survey Week Logic

NFP reference week = pay period containing the **12th of the month**.

1. Find the 12th of the month
2. Find the Sunday of that week (survey week start)
3. Compute gap to previous month's survey week:

$$\texttt{weeks} = \lfloor (\text{survey}_t - \text{survey}_{t-1})/7 \rfloor$$

Result: **4 or 5 weeks**. A 5-week interval allows more accumulated job growth/loss, systematically inflating NSA counts.

Other Features

- `is_jan`: BLS seasonal factor refresh
- `is_july`: Mid-year benchmark revision
- `year`: Secular trend

Dynamic Feature Selection (2-Pass)

When master snapshots store ALL features (`mode: "all_features"`)

Dynamic reselection is the sole feature selection path during the backtest.

Pass 1: Per-Source (Stages 0,2,4,5)

```
for source in [FRED_NSA, FRED_SA,
               FRED_Exog, Unifier,
               ADP, NOAA, Prosper]:
    survivors[source] = run_fs(
        X_train[source_cols],
        y_train,
        stages=(0, 2, 4, 5)
    )
```

Pass 2: Global (Stages 0,1,2,4)

```
all_survivors = union(survivors.values())
all_survivors += target_derived_features
all_survivors += calendar_features

final = run_fs(
    X_train[all_survivors],
    y_train,
    stages=(0, 1, 2, 4),
    max_features=50 # hard cap
)
```

Parameters

Reselection frequency: every 24 months (configurable). NaN eval window: post-2010. Max NaN rate: 20%. Sample weighting: uniform (no recency decay) — selects features with durable long-term signal.

Dynamic Feature Selection: All-Features Mode

All-Features Mode

Master snapshots store **all 15,497 lean features**. No ETL-time pruning. Feature selection is entirely deferred to the 2-pass dynamic reselection during the backtest.

Advantages:

- Every feature gets a fresh evaluation each window
- No permanent information loss from ETL decisions
- Adapts to structural regime changes

Uniform Sample Weighting

Equal weight across all history (no recency decay). Selects features with **durable, long-term predictive power** rather than recent-month noise.

$$w_i = 1 \quad \forall\, i \quad \text{(uniform)}$$

Jaccard stability: 0.976–0.979 between windows.

Branch-Target: Dynamics Composite (SA)

Multi-signal ranking for variance capture:

$$\begin{aligned} S = {} & 0.25 \cdot |\rho(X, y)| && \text{Level} \\ & + 0.25 \cdot |\rho(\Delta X, \Delta y)| && \text{Diff} \\ & + 0.15 \cdot \tanh\left(\frac{|\mu_\uparrow - \mu_\downarrow|}{\sqrt{\sigma^2}}\right) && \text{Direction} \\ & + 0.20 \cdot |\rho(|\Delta X|, |\Delta y|)| && \text{Magnitude} \\ & + 0.10 \cdot \text{sign_agree} && \text{Sign} \\ & + 0.05 \cdot \text{tail_align} && \text{Tail} \end{aligned}$$

NSA: uses simple `weighted_corr` (top 8). SA: uses dynamics composite (top 20).

Sample Weighting: Exponential Decay + Tail Awareness

Exponential Decay

Recent observations weighted higher:

$$w_i = \exp\left(-\frac{\ln 2 \cdot d_i}{h}\right)$$

where:

- $d_i = (\text{target_month} - \text{date}_i)/30.44$ (months)
- $h \in [12, 120]$ months (tuned by Optuna)
- Normalized: $w \leftarrow w/\bar{w}$

Interpretation: With $h = 60$ months, data from 5 years ago has weight $= 0.5$; data from 10 years ago has weight $= 0.25$.

Tail-Aware Boosting (Variance Targets)

Prevents the model from ignoring large moves:

$$\begin{aligned} m_i &= 1.0 \quad \text{(base)} \\ \text{if } |y_i| \geq Q_{0.80}(|y|): &\quad m_i *= 1.35 \\ \text{if } |\Delta y_i| \geq Q_{0.80}(|\Delta y|): &\quad m_i *= 1.35 \\ m_i &\leftarrow \text{clip}(m_i,\ 1.0,\ 2.50) \\ w_i^{\text{final}} &= w_i^{\text{decay}} \cdot m_i \end{aligned}$$

Observations with both extreme level AND extreme change get up to $2.5\times$ weight boost.

LightGBM Training: CV + Final Model

Training Process

```
# Phase 1: 5-fold TimeSeriesSplit CV
for fold in TimeSeriesSplit(n_splits=5):
    train on earlier data
    validate on later data
    accumulate OOF residuals

# Phase 2: Final model
train_size = 85% of data
X_train, X_val = split(X, 0.85)
model = lgb.train(
    params,
    train_data,
    num_boost_round=1000,
    early_stopping=50,
    valid_sets=[val_data]
)
# Extract gain importance + residuals
```

Why LightGBM?

- **Native NaN handling:** Split-finder tries both branches for missing values, choosing whichever reduces loss more
- Critical for staggered datasets where FRED starts ∼1948 but Prosper starts ∼2009
- No imputation = no artificial patterns
- `inf` replaced with NaN (LightGBM handles NaN but not inf)

Default Parameters

objective	regression
metric	mae
learning_rate	0.03
num_leaves	31
max_depth	6
feature_fraction	0.8
bagging_fraction	0.8

Hyperparameter Tuning: Leakage-Safe Optuna

Architecture

- **Inner TimeSeriesSplit** (5 folds) within each outer expanding window step
- Outer backtest: data $< t$
- Inner CV: further splits into train/val
- No future data ever leaks

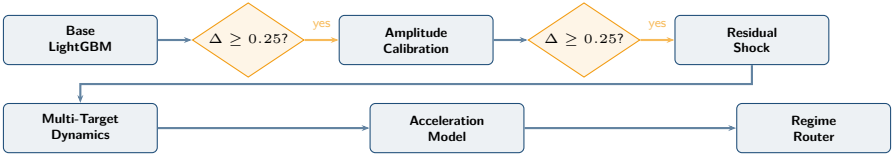
Optimization Config

Sampler	TPE (seed=42)
Pruner	Median (10 startup, 20 warmup)
Budget	25 trials, 300s timeout
Frequency	Every 12 backtest months
Warm-start	Previous best $\rightarrow$ Trial 0

Search Space

Parameter	Range	Scale
learning_rate	[0.005, 0.15]	Log
num_leaves	[15, 127]	Linear
max_depth	[3, 8]	Linear
min_data_in_leaf	[1, 50]	Linear
feature_fraction	[0.4, 1.0]	Linear
bagging_fraction	[0.5, 1.0]	Linear
λ_{L1}	$[10^{-8}, 10]$	Log
λ_{L2}	$[10^{-8}, 10]$	Log
half_life (months)	[12, 120]	Linear
huber_delta	[25, 500]	Linear

Variance Enhancement: 5-Stage Sequential Stack



Gate Rule

Each stage is kept **only if** it improves the composite validation score by ≥ 0.25. Otherwise, the prediction from the previous stage is carried forward.

Enhancement Sequences

- **NSA:** Full stack (amplitude $\rightarrow$ shock $\rightarrow$ dynamics $\rightarrow$ acceleration $\rightarrow$ regime)
- **SA:** Enhancement exempt (base LightGBM predictions—amplitude calibration tested but degraded recent performance)

Enhancement Details: Amplitude, Shock, Dynamics

A: Amplitude Calibration

$$\hat{y}_{\text{cal}} = a + b \cdot \hat{y}$$

where $(a, b) = \text{polyfit}(\hat{y}_{\text{val}},\ y_{\text{val}},\ 1)$

Slope clamped: $b \in [0.50, 3.00]$

Min samples: 12

Purpose: Correct systematic under/over-prediction of magnitude.

B: Residual Shock

Separate shallow LightGBM on residual errors:

- max_depth: 3
- num_leaves: 15
- 200 boosting rounds

If residuals have structure (e.g., larger during high-VIX), this captures it.

$$\hat{y}_{\text{shock}} = \hat{y}_{\text{cal}} + f_{\text{shock}}(X)$$

C: Multi-Target Dynamics

Three sub-models:

- **Magnitude:** $|\Delta\hat{y}|$
- **Direction:** $P(\Delta y > 0)$
- **Level:** Current best $\hat{y}$

$$\delta_c = 0.7\delta_{\text{mag}} + 0.3\delta_{\text{cur}}$$
$$c = |p - 0.5|$$
$$b = \min(0.8c,\ 1)$$
$$\delta_f = (1-b)\delta_c + b\delta_{\text{dir}}$$

Min confidence: $|p - 0.5| > 0.12$

Enhancement Details: Acceleration & Regime Router

D: Acceleration Model

Train on the "change of change" target:

$$y_{\text{accel}}^{(t)} = y^{(t)} - y^{(t-1)}$$

Separate shallow LightGBM (depth=3, leaves=15, 200 rounds).

Reconstruction:

$$\hat{y}^{(t)} = y^{(t-1)} + \hat{y}_{\text{accel}}^{(t)}$$

Motivation: Captures momentum dynamics that the level model misses. If NFP has been accelerating (increasing MoM), this model detects it.

E: Regime Router

Partition training data by target volatility:

- Low-vol: $|y| < Q_{0.75}(|y|)$
- High-vol: $|y| \geq Q_{0.75}(|y|)$

Train separate expert LightGBMs for each regime.

Router: Logistic classifier predicts $P(\text{high-vol})$.

Soft blend:

$$\hat{y} = (1 - p_h) \cdot \hat{y}_{\text{low}} + p_h \cdot \hat{y}_{\text{high}}$$

Min 20 samples per regime required.

Variance Capture Metrics

The Problem: Variance Collapse

Standard metrics (RMSE, MAE) can mask a critical failure: the model predicts the general trend but **flattens month-to-month amplitude**. The prediction series has lower standard deviation than the actual.

Metric	Formula	Target	Interpretation
STD Ratio	$\sigma(\hat{y})/\sigma(y)$	1.0	< 1 means flattening
Diff STD Ratio	$\sigma(\Delta\hat{y})/\sigma(\Delta y)$	1.0	MoM acceleration amplitude
ρ_{level}	$\text{corr}(y, \hat{y})$	> 0.8	Overall trend following
ρ_{diff}	$\text{corr}(\Delta y, \Delta\hat{y})$	> 0.6	Change-of-change tracking
Diff Sign Acc	$\mathbb{E}[\text{sign}(\Delta y) = \text{sign}(\Delta\hat{y})]$	> 0.65	Direction of change
Tail MAE	$\mathbb{E}[\lvert e\rvert; \lvert y\rvert \geq Q_{0.75}]$	minimize	Error on large moves
Extreme Hit	$P(\lvert\hat{y}\rvert \geq Q_{0.90} \mid \lvert y\rvert \geq Q_{0.90})$	> 0.60	Extreme event recall

Composite Objective Function

Used by Optuna Tuning (`objective_mode = 'composite'`) for SA Targets

$$
\begin{aligned}
\mathcal{L}_{\text{composite}} = \text{MAE} \\
&+ 25.0 \cdot |1 - \text{STD Ratio}| \\
&+ 25.0 \cdot |1 - \text{Diff STD Ratio}| \\
&+ 0.20 \cdot \text{Tail MAE} \\
&+ 20.0 \cdot (1 - \rho_{\text{diff}}) \\
&+ 12.0 \cdot (1 - \text{Diff Sign Accuracy}) \\
&+ 15.0 \cdot (1 - \text{Acceleration Accuracy}) \\
&+ 10.0 \cdot (1 - \text{Directional Accuracy})
\end{aligned}
$$

Variance Promotion Gates (SA)

Gate	Minimum

Directional Accuracy

Two complementary metrics:

Prediction Intervals & Baselines

Non-Parametric Confidence Intervals

No Gaussian assumption. Purely empirical from historical OOS residuals:

$$\mathrm{CI}_\alpha = \left[\hat{y} + Q_{\alpha/2}(r),\ \ \hat{y} + Q_{1-\alpha/2}(r)\right]$$

where $r = \{y_i - \hat{y}_i\}$ are historical residuals.

Coverage levels: 50%, 80%, 95%

Requires ≥ 10 residuals. For forward predictions, uses up to last 36 OOS residuals.

Naive Baselines

Computed **per backtest step** using only training data:

Last-y: $\hat{y}_{\text{base}} = y_{t-1}$ (random walk)

Rolling Mean: $\hat{y}_{\text{base}} = \frac{1}{6}\sum_{k=1}^{6} y_{t-k}$

Keep Rule

If trailing 12-month MAE exceeds best baseline:

- Action: `skip_save` (don't deploy model)
- Tolerance: 0.0 (must beat baseline)
- Window: 12 months trailing OOS

Model Variants & Output Structure

2 Model Variants (Default)

ID	Target	Purpose
`nsa_first_revised`	NSA	Once-revised NSA MoM
`sa_first_revised`	SA	Once-revised SA MoM

Revised target: MoM from M+1 FRED snapshot (once-revised).

`predict_nfp_mom()` enforces `operational_available_date` checks at inference.

Consensus Anchor (Post-Training)

- **Kalman Fusion:** State-space model fusing consensus + model
- **Accel Override:** Consensus base + model momentum
- Both PIT-safe (min 12 months history)

Output Artifacts (`_output/`)

Folder	Contents
`NSA_prediction/`	Backtest CSV, predictions PNG, SHAP, feature importance, summary stats
`SA_prediction/`	Same structure
`NSA_plus_adjustment/`	NSA + exp-weighted seasonal adjustment vs actual SA
`Predictions/`	Forward predictions with 50/80/95% CIs
`models/lightgbm_nfp/`	Saved models + comparison scorecard (CSV + HTML)
`Archive/`	Timestamped snapshots

Running the Pipeline

Full Pipeline

```python
# End-to-end
python run_full_project.py

# Fresh re-download
python run_full_project.py --fresh

# Individual stages
python run_full_project.py --stage data
python run_full_project.py --stage train
python run_full_project.py --stage train --no-tune

# Skip sources
python run_full_project.py --skip noaa,prosper
```

Direct Training

```python
# All variants + scorecard
python Train/train_lightgbm_nfp.py --train-all

# Single variant
python Train/train_lightgbm_nfp.py --train \
  --target sa --release first

# Production inference
python scripts/predict_next_nfp.py --target nsa
```

Environment Overrides

```python
# Fast feature selection
NFP_FS_STAGES="0,1,4" python run_full_project.py

# Enable performance profiling
NFP_PERF=1 python run_full_project.py
```

Economic Shock Handling & Reproducibility

COVID Winsorization

- Spring 2020 extremes clipped to non-COVID quantiles (1st/99th percentile)
- Applied **per-fold** during backtest (not globally)
- Model doesn't "know" COVID happened until it reaches March 2020
- Preserves directional information

SymLog Transform

$$\text{symlog}(x) = \text{sign}(x) \cdot \ln(1 + |x|)$$

- Handles both positive and negative values
- Compresses extreme kurtosis
- $\text{symlog}(0) = 0$ (preserves zero)
- Inverse: $\text{sign}(y) \cdot (e^{|y|} - 1)$

Post-1990 Data Floor

`DATA_START_FLOOR = 1990-01-01`

Pre-1990 data is extremely sparse for non-FRED sources and degrades feature selection quality.

Reproducibility

- **Seeds:** LightGBM `random_state=42`, Optuna TPE `seed=42`
- **Determinism:** Expanding-window splits are strictly chronological, no shuffling
- **Tracking:** Perf profiling JSONs in `_temp/perf/`
- **Archiving:** Output snapshots in `_output/Archive/`
- **CI:** pytest + ruff + mypy across Python 3.10–3.12

Consensus Anchor: Kalman Filter Fusion

State-Space Model (3-Channel Information Filter)

State: x_t = true SA NFP MoM (random walk)

$$\text{Prior:} \quad x_t^- = x_{t-1}, \quad P_t^- = P_{t-1} + Q$$

$$\text{Update:} \quad P_t = \left(\frac{1}{P_t^-} + \frac{1}{R_c} + \frac{1}{R_m} + \frac{w_a}{R_a} \right)^{-1}$$

$$x_t = P_t \left(\frac{x_t^-}{P_t^-} + \frac{z_c}{R_c} + \frac{z_m}{R_m} + \frac{w_a z_a}{R_a} \right)$$

where:

- z_c = consensus prediction, R_c = trailing consensus error variance
- z_m = model channel (R_m = trailing error variance):
  - Model A: SA Direct LightGBM
  - Model B: NSA+Adjustment
- z_a = NSA+Adj acceleration channel (Model A only)
- w_a = NSA weight scale (Optuna-tuned)

Optuna Tuning (Composite Objective)

$$\mathcal{L} = \text{MAE} - 50 \cdot \text{AccelAcc} - 30 \cdot \text{DirAcc}$$

Tunable parameters:

- `trailing_window` $\in [6, 36]$ months
- `nsa_weight_scale` $\in [0.1, 3.0]$

Tuned values: tw $= 28$, $w_a = 1.84$

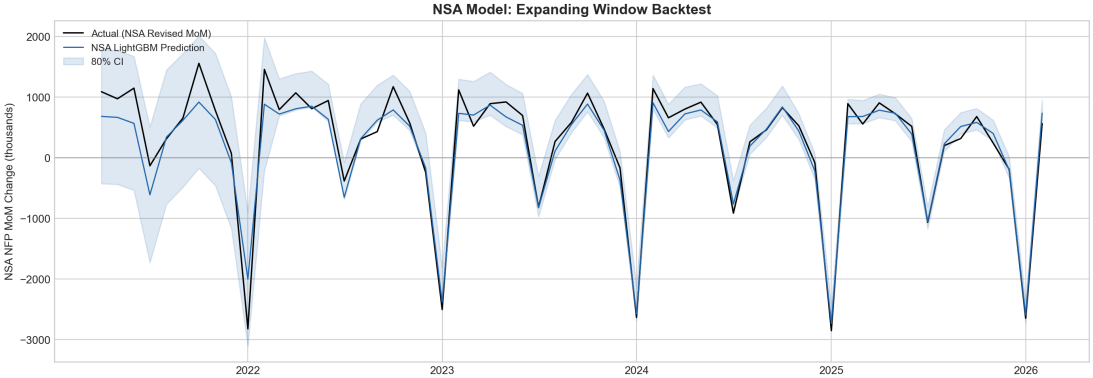
AccelOverride (Direction Correction)

Majority vote on acceleration direction using consensus + SA blend + NSA+Adj signals. If majority disagrees with consensus direction:

$$\hat{y} = y_{t-1} + \text{dir}_{\text{majority}} \cdot |z_c - y_{t-1}|$$

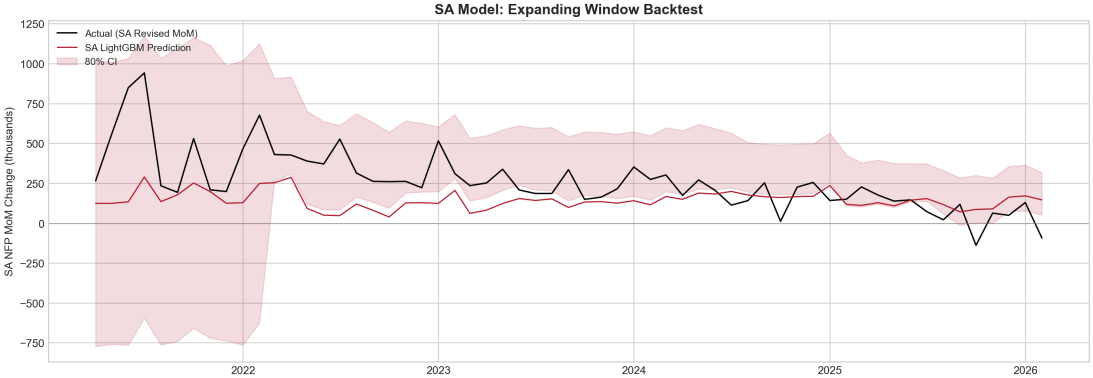
Tuned: $\kappa = 0.40$, mode = "consensus"

Backtest Results: NSA Model (60-Month OOS)



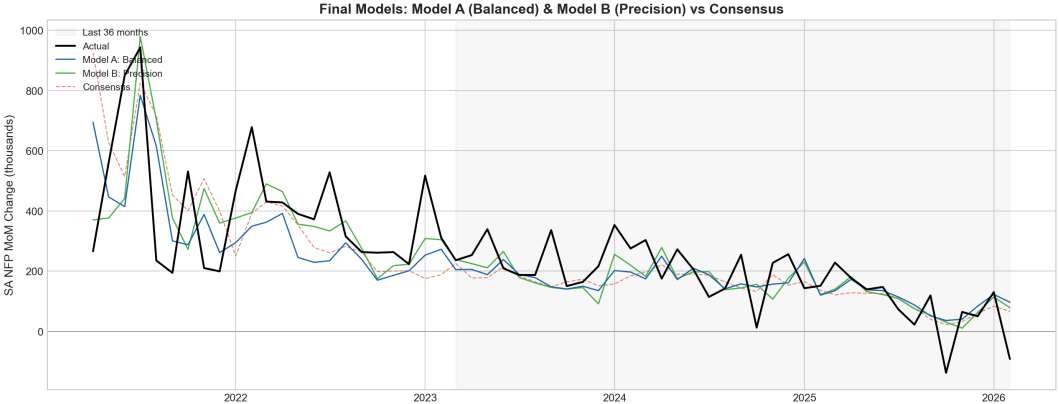
- 60-month expanding window backtest (2021-04 to 2026-02), model retrained from scratch each month
- NSA MAE: 241.6K (vs baseline last-y: 911.7K, rolling mean: 701.1K)
- 100% directional accuracy, 88.2% acceleration accuracy on NSA target
- Full enhancement stack active (amplitude, shock, dynamics, acceleration, regime)

Backtest Results: SA Model (60-Month OOS)



- SA model uses separately selected features from SA master snapshots
- Dynamic reselection every 24 months with uniform sample weighting (no recency bias)
- 80 features selected per window (50 snapshot + branch-target + calendar + revision)
- SA enhancement: exempt from variance stack (base LightGBM predictions used directly)

Two Final Models: Balanced vs Precision



Both models use consensus-anchored Kalman fusion. They differ in which model signals feed the filter:

Model A: Balanced (Blue)

3-channel: consensus + SA Direct + NSA+Adj
Best **acceleration accuracy** (54.3% recent)

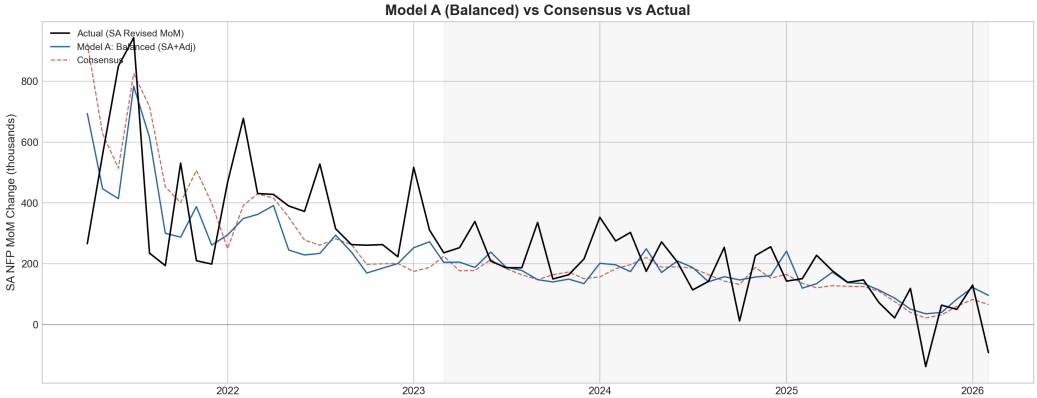
Model B: Precision (Green)

2-channel: consensus + NSA+Adj only
Best **MAE** and **RMSE** (95.5 / 135.5)

Model Comparison: Full Period & Recent

Model	Full 59 Months MAE	RMSE	Accel	Dir	Last 36 Months MAE	RMSE	Accel	Dir
Model A (Balanced)	105.9	148.2	**56.9%**	96.6%	67.1	84.0	**54.3%**	94.4%
Model B (Precision)	**95.5**	**135.5**	53.4%	96.6%	67.6	**82.3**	40.0%	94.4%

Model A: Balanced — Best for Acceleration



Strengths

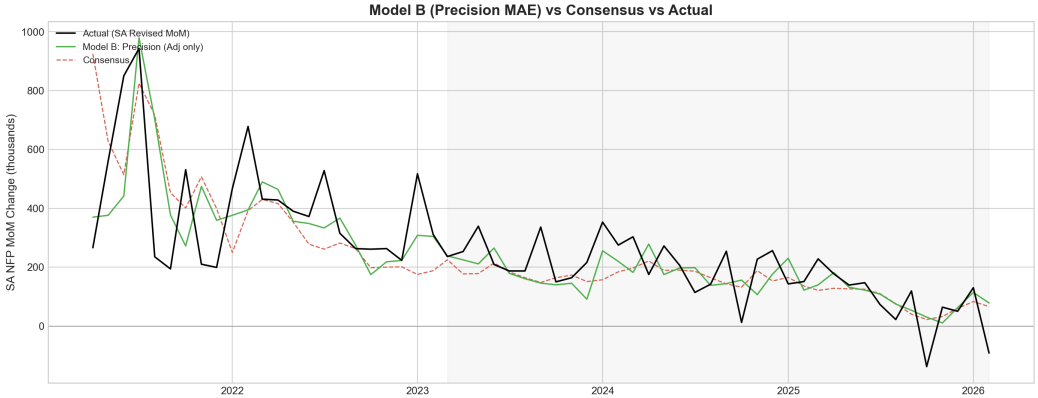
- **54.3% acceleration accuracy** on last 36m (vs 42.9% consensus: +26%)
- 56.9% full-period acceleration (+27% vs consensus)

Architecture

Kalman filter with 3 channels:

1. Consensus (level anchor)
2. SA Direct LightGBM (variance signal)

Model B: Precision — Best for MAE/RMSE



Strengths

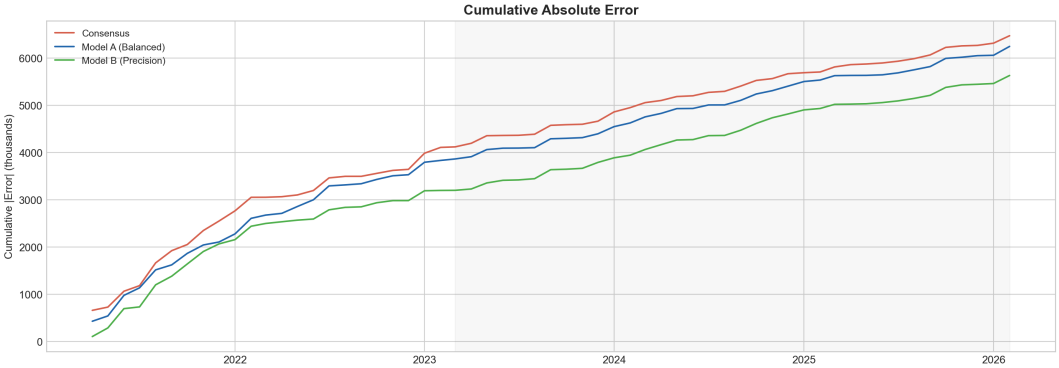
- **MAE 95.5** (-13.0% vs consensus)
- **RMSE 135.5** (-18.4% vs consensus)
- Lowest cumulative error over full period

Architecture

Kalman filter with 2 channels:

1. Consensus (level anchor)
2. NSA+Adjustment (acceleration + level)

Cumulative Error: Both Models vs Consensus



Key Insight

Both models accumulate less total error than consensus. Model B (green) has the lowest cumulative error; Model A (blue) tracks closely while capturing acceleration dynamics better.

When to Use Which

- **Model A:** When catching turning points and momentum shifts matters most (trading, risk)
- **Model B:** When minimizing absolute prediction error matters most (forecasting, planning)

Feature Selection & Post-Training Fusion

Dynamic Reselection Config

Parameter	Value
Reselection interval	36 months
Sample weighting	Uniform
Max features (Pass-2)	80
Total in snapshots	15,497
Optuna objective	Composite
Optuna trials	25 per window

Top SA Features

1. `CCSA_monthly_avg_diff`
2. `NFP_Consensus_Mean_chg_3m`
3. `CB_Consumer_Confidence_chg_12m`
4. `nfp_sa_accel_rolling_3m`
5. `Financial_Stress_monthly_avg`

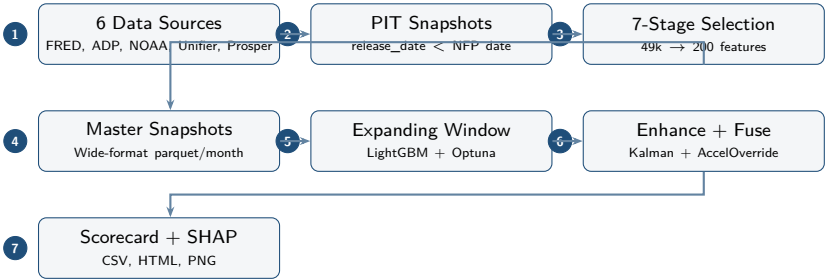
Post-Training Chain

1. **NSA LightGBM** → NSA predictions (full enhancement stack)
2. **SA LightGBM** → SA predictions (enhancement exempt, with NSA accel features)
3. **Predicted Adjustment:** NSA pred + exp-weighted seasonal factor → NSA+Adj (SA space)
4. **Kalman Fusion:**
   - Model A: cons + SA + NSA+Adj
   - Model B: cons + NSA+Adj
5. **Output:** Two predictions per month; user selects based on priority

Key Design Principles

1. **Point-in-Time Above All:** Every feature satisfies `release_date` $<$ `nfp_release_date`. No same-day leakage. Per-source PIT fixes for FRED vintage, Unifier bugs, NOAA lags, weekly windowing.
2. **Native NaN Handling:** LightGBM handles missing values mathematically. No imputation, no forward-fill. Different start dates across sources are tolerated.
3. **Regime Awareness:** Feature selection tests stability across 6 macroeconomic regimes. Exponential recency weighting prioritizes recent signal.
4. **Variance Capture:** Composite objective prevents "variance collapse." 5-stage enhancement stack (amplitude, shock, dynamics, acceleration, regime). Tail-aware sample weighting for extreme events.
5. **Expanding Window Discipline:** Model retrained from scratch each month. No K-Fold. COVID winsorization per-fold. Baseline keep-rule gating prevents deployment of models worse than naive heuristics.
6. **Efficiency at Scale:** $p \gg n$ datasets (15k features, $\sim$430 rows) handled via random subspace selection, chunked clustering, vectorized operations, joblib parallelism, and aggressive caching.

Pipeline Summary: Data → Prediction



Results: Both Models Beat Consensus

59-month OOS backtest (2021–2026):

- **Model A:** AccelAcc 56.9% (+27%), MAE 105.9
- **Model B:** MAE 95.5 (−13%), RMSE 135.5 (−18%)
- Both beat consensus on full period
- User selects based on priority

Production Output

- Two predictions generated per month
- Model A: best for momentum/acceleration
- Model B: best for MAE/RMSE precision
- Full comparison scorecard + SHAP

Thank You

Questions?

Dhruv Kohli · March 16, 2026